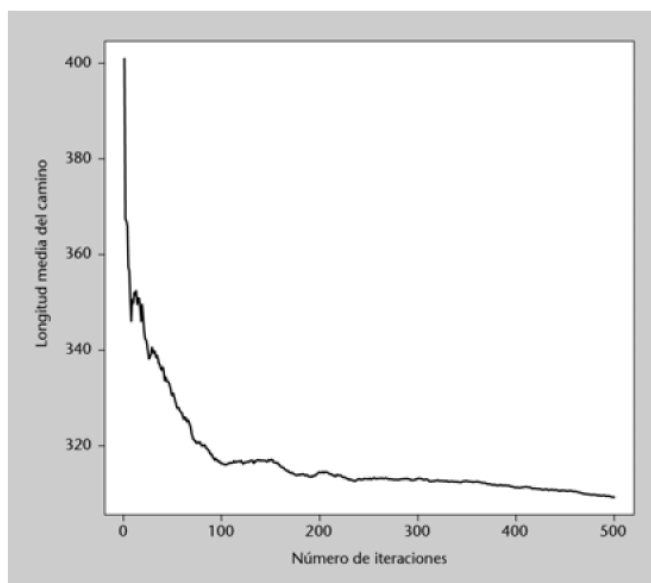


Figura 58. Efecto del número de iteraciones en la longitud media del camino



5.4. Código fuente en Python

Código 45: Algoritmo de la colonia de hormigas

```

1  import random, sys, math
2
3  # Nota: en lugar de matrices se usan listas de listas
4
5  # Genera una matriz de distancias de nCiudades x nCiudades
6  def matrizDistancias(nCiud, distanciaMaxima):
7      matriz = [[0 for i in range(nCiud)] for j in range(nCiud)]
8
9      for i in range(nCiud):
10         for j in range(i):
11             matriz[i][j] = distanciaMaxima*random.random()
12             matriz[j][i] = matriz[i][j]
13
14     return matriz
15
16 # Elige un paso de una hormiga, teniendo en cuenta las distancias
17 # y las feromonas y descartando las ciudades ya visitadas.
18 def eligeCiudad(dists, ferom, visitadas):
19     # Se calcula la tabla de pesos de cada ciudad
20     listaPesos = []
21     disponibles = []
22     actual = visitadas[-1]

```

```

23
24     # Influencia de cada valor (alfa: feromonas; beta: distancias)
25     alfa = 1.0
26     beta = 0.5
27
28     # El parámetro beta (peso de las distancias) es 0.5, alfa=1.0
29     for i in range(len(dists)):
30         if i not in visitadas:
31             fer = math.pow((1.0 + ferom[actual][i]), alfa)
32             peso = math.pow(1.0/dists[actual][i], beta) * fer
33             disponibles.append(i)
34             listaPesos.append(peso)
35
36     # Se elige aleatoriamente una de las ciudades disponibles,
37     # teniendo en cuenta su peso relativo.
38     valor = random.random() * sum(listaPesos)
39     acumulado = 0.0
40     i = -1
41     while valor > acumulado:
42         i += 1
43         acumulado += listaPesos[i]
44
45     return disponibles[i]
46
47
48 # Genera una "hormiga", que elegirá un camino teniendo en cuenta
49 # las distancias y los rastros de feromonas. Devuelve una tupla
50 # con el camino y su longitud.
51 def eligeCamino(distancias, feromonas):
52     # La ciudad inicial siempre es la 0
53     camino = [0]
54     longCamino = 0
55
56     # Elegir cada paso según la distancia y las feromonas
57     while len(camino) < len(distancias):
58         ciudad = eligeCiudad(distancias, feromonas, camino)
59         longCamino += distancias[camino[-1]][ciudad]
60         camino.append(ciudad)
61
62     # Para terminar hay que volver a la ciudad de origen (0)
63     longCamino += distancias[camino[-1]][0]
64     camino.append(0)
65
66     return (camino, longCamino)
67
68 # Actualiza la matriz de feromonas siguiendo el camino recibido

```

```

69 def rastrosFeromonas(feromonas, camino, dosis):
70     for i in range(len(camino) - 1):
71         feromonas[camino[i]][camino[i+1]] += dosis
72
73 # Evapora todas las feromonas multiplicándolas por una constante
74 # = 0.9 (en otras palabras, el coeficiente de evaporación es 0.1)
75 def evaporaFeromonas(feromonas):
76     for lista in feromonas:
77         for i in range(len(lista)):
78             lista[i] *= 0.9
79
80 # Resuelve el problema del viajante de comercio mediante el
81 # algoritmo de la colonia de hormigas. Recibe una matriz de
82 # distancias y devuelve una tupla con el mejor camino que ha
83 # obtenido (lista de índices) y su longitud
84 def hormigas(distancias, iteraciones, distMedia):
85     # Primero se crea una matriz de feromonas vacía
86     n = len(distancias)
87     feromonas = [[0 for i in range(n)] for j in range(n)]
88
89     # El mejor camino y su longitud (inicialmente "infinita")
90     mejorCamino = []
91     longMejorCamino = sys.maxint
92
93     # En cada iteración se genera una hormiga, que elige un camino,
94     # y si es mejor que el mejor que teníamos, deja su rastro de
95     # feromonas (mayor cuanto más corto sea el camino)
96     for iter in range(iteraciones):
97         (camino, longCamino) = eligeCamino(distancias, feromonas)
98
99         if longCamino <= longMejorCamino:
100             mejorCamino = camino
101             longMejorCamino = longCamino
102
103         rastrosFeromonas(feromonas, camino, distMedia/longCamino)
104
105     # En cualquier caso, las feromonas se van evaporando
106     evaporaFeromonas(feromonas)
107
108
109     # Se devuelve el mejor camino que se haya encontrado
110     return (mejorCamino, longMejorCamino)
111
112
113 # Generación de una matriz de prueba
114 numCiudades = 10

```

```

115 distanciaMaxima = 10
116 ciudades        = matrizDistancias(numCiudades, distanciaMaxima)
117
118 # Obtención del mejor camino
119 iteraciones = 1000
120 distMedia   = numCiudades*distanciaMaxima/2
121 (camino, longCamino) = hormigas(ciudades, iteraciones, distMedia)
122 print("Camino:_", camino)
123 print("Longitud del camino:_", longCamino)

```

6. Optimización con enjambres de partículas

Supongamos que se nos plantea buscar el mínimo (o máximo) de una función matemática cualquiera, como por ejemplo

$$f(x, y) = x^2(4 - 2,1x^2 + \frac{x^4}{3}) + xy + y^2(-4 + 4y^2) \quad (59)$$

dentro de un intervalo determinado, en este caso $[(-10, -10), (10, 10)]$. Se trata de un ejemplo puramente ilustrativo, pues f es una función que se puede derivar fácilmente y por tanto, es posible encontrar los mínimos de forma analítica. Concretamente, hay dos mínimos globales, en $(0,09, -0,71)$ y en $(-0,09, 0,71)$, en los que la función toma el valor de $-1,0316$.

El método de optimización con enjambres de partículas (en inglés, *particle swarm optimization*) debe su nombre a que utiliza un conjunto de soluciones candidatas que se comportan como partículas moviéndose por un espacio: el espacio de soluciones. El movimiento de las partículas se determina por su velocidad, que depende de la posición del mejor punto que ha encontrado cada una de ellas en su recorrido y del mejor punto encontrado globalmente por el enjambre.

Vale la pena mencionar el origen de este método⁷, pues curiosamente surgió a partir de un trabajo que simulaba el comportamiento de entidades sociales como rebaños de animales. Analizando el comportamiento de esa simulación, y simplificando un poco las reglas que la

⁷ El método de optimización con enjambres de partículas se describió en J. Kennedy; R. Eberhart (1995). "Particle Swarm Optimization". IEEE International Conference on Neural Networks.